University of Plymouth

School of Engineering, Computing, and Mathematics

COMP3000
Computing Project
2025/2026

Hook Line and Slinger

Michael Hooper

10852833

BSc (Hons) Computer Science (Games Development) (7394)

Acknowledgements

First, I would like to express gratitude to my supervisor, Dr Ji-Jian Chin for providing invaluable feedback and help with various aspects of this project. I am also grateful to the other lecturers at the University of Plymouth for helping with clarifying certain parts of the project brief and providing other help when needed.

Additionally, I would like to extend sincere thanks to all my fellow students who attended the user testing session hosted at the university on the 25th of March 2026 for providing me with suitable feedback for the expansion and improvement of my game. Finally, I would be remiss in not mentioning my friends and family for giving me support during the production of my project.

Abstract

Hook Line and Slinger is a video game designed to fill the niche area of the market that is platformer games that utilise physics to provide complex and engaging movement for players. The primary objectives of this project are to fully develop at least one playable level for users to play through from the start to completion, and to create a fun grappling hook system that as many players as possible can deeply enjoy. Some goals that need to be completed include, but are not limited to: implementing a grappling hook system; creating enemies that fire projectiles in the direction of the player and a boss for each level that tests a different part of the player's movement kit; models and animations for all entities in the world; and a working title screen and pause menu.

Contents

1 Introduction 4
2 Background, Objectives and Deliverables 5
2.1 Background 5
2.2 Objectives 6
2.3 Deliverables 7
3 Market Research 7
3.1 Hollow Knight: Silksong 8
3.2 Rusted Moss 9
3.3 Flinthook 10
3.4 Cuphead 11
3.5 The King's Bird 12
3.6 Mega Man 11 13
4 Legal, Social, Ethical, and Professional Issues 14
4.1 Legal Issues 14

4.2 Social Issues 15
4.3 Ethical Issues 15
4.4 Professional Issues 16
5 Method of Approach 17
5.1 Development Methodology 17
5.2 System Architecture 18
5.3 Game Design 19
5.4 Tools and Technologies 20
5.5 Project Planning 21
5.6 Risk Management 21
6 End-project Report 22
6.1 Evaluation of Objectives 22
6.2 Changes During the Project 23
6.3 Testing 25
6.3.1 Level Completion Rate 25
6.3.2 Player Previous Experience 25
6.3.3 Alternate Bait Usage 25
6.3.4 Likert Scale Questions 26
6.3.5 Difficulty Rating 26
6.3.6 Written Feedback 26
6.4 Overall Project Success 27
7 Project Reflections 27
7.1 Reflection on Development Process 27
7.2 Reflection on Technical Decisions 28
7.3 Reflection on Personal Performance 29
7.4 Reflection vs Market Research 29
7.5 What Could Be Done Differently 30
8 Conclusion 30
9 References/Bibliography 31
9.1 Other game engines 31
9.1.1 2D in Unreal Engine 31
9.1.2 2D in Godot 31
9.2 Game research 31

9.2.1 Hollow Knight: Silksong 31
9.2.2 Rusted Moss 31
9.2.3 Flinthook 32
9.2.4 Cuphead 32
9.2.5 The King's Bird 32
9.2.6 Mega Man 11 32
9.2.7 Using the Clawline in Hollow Knight Silksong 32
9.2.8 Level select in Mega Man 11 32
9.3 LSEP 32
9.3.1 Steam's player base 32
9.3.2 Steam Partner Program 32
9.3.3 Unity Pricing 33
9.3.4 PEGI ratings 33
9.4 Tools 33
9.4.1 Unity Input System 33
9.4.2 Piskel 33
10 Appendices 33
10.1 User Guide 33
10.2 Project Planning & Agile Evidence 34
10.2.1 Gantt Chart 34
10.2.2 Trello board with all sprints 34
10.3 Technical Design & Architecture 36
10.4 Code Samples 41
10.4.1 Player Movement 41
10.4.2 Projectiles 43
10.4.3 Game Control 44
10.4.4 Boss 45
10.5 User Testing / Feedback 45
10.5.1 Level Completion Rate 45
10.5.2 Player Previous Experience 46
10.5.3 Alternate Bait Usage 46
10.5.4 Chance of Replayability 46
10.5.5 Likert Scale Questions 47

10.5.6 Difficulty Rating 47
10.5.7 Written Feedback 48
10.6 Asset & Resource List 50
10.7 Risk Assessment 50
10.8 AI Declaration 53

WORD COUNT: **10,536**

https://github.com/MichaelHooper36/COMP3000-Final-Year-Project

1 Introduction

In the world of 2D platformers, there are a substantial number that utilise various mechanics to make the player move quickly from one side of a map to the other and significantly fewer that utilise grappling hooks and other rope-based systems. Most of these can feel floaty or are constrained to a small room and while they can feel fun to play through, none of them provide an experience that utilises physics in a unique or realistic way within the bounds of 2D.

Hook Line and Slinger is a physics-based 2D platformer shooter where users play as a sushi-chef-turned-fisherman on the hunt for the rarest and most delicious fish in the ocean. Users will be given the ability to move around linear levels of terrain using the character's main body for running and its fishing rod as a grappling hook for grappling to specific points around each level. Additionally, there will be fish around the level that will try to defeat the player unless they choose to fight back with their various bait.

In each level, there are multiple types of bait available to the player with different amounts of damage and ways of travelling in the air. These can be found from either hidden areas in each level or from defeating enough fish in a specific level.

At the end of each level, players will have the opportunity to test their skills against an elite or boss enemy that is designed to be significantly more difficult that the fish while also testing the players' understanding of certain mechanics. If the player defeats the boss, they will unlock the ability to continue to the next level and repeat this cycle with new and exciting enemies and movement mechanics.

This game will be primarily aimed at speed-runners or people who enjoy playing 2D shooters like Mega Man and Cuphead or people who enjoy 2D platformers like Hollow Knight; however, this game will be aimed for people of all ages and experience in games. The difficulty will be medium and encourages trial and error so that newer players can complete levels with enough time spent in each level and then the more experienced players can try to run through levels to get as fast a time as possible.

As shown below in the list of objectives, this project will focus on the creation of the first level within the span of seven months using agile development and an iterative process, with a working character model for players to run around the level, defeating various enemies and a boss at the end, with a grappling system that is unique and different from those in other 2D platformers and a low-poly pixel art aesthetic.

In this report, the following will be discussed: all of the objectives and deliverables that need to be completed for the minimum viable product(MVP); market research on other games and how Hook Line and Slinger differs from these games in substantial ways; the legal, social, ethical, and professional issues that could appear with the production of this game; the methodology of how the game will be developed; and the report and reflections that come at the end of the project's development.

2 Background, Objectives and Deliverables

2.1 Background

The main idea of Hook Line and Slinger is a 2D platformer with linear progression and exploration where the player moves the character from start to finish, defeating as many of the optional minor enemies as desired before defeating a boss at the end to unlock new mechanics and the ability to play the subsequent level.

The main loop and replay-ability of Hook Line and Slinger comes from two main aspects: the variety of bait and the timer for each level. In the game, the player character has bait which can be thrown at fish to damage them and reduce their health to zero. Each piece of bait should function differently to provide variety to gameplay, like a type of bait that moves in the air in a specific pattern or a type of bait that offers extra movement abilities. In each level of Hook Line and Slinger there is a timer in the top right corner of the screen which stops when the boss is defeated, and the best time will be saved on the level select screen. This will provide an incentive to replay levels so that players can obtain a faster time. After a player has obtained certain bait, they might be able to use that bait on earlier levels, providing a much faster time in some instances. This will help with the incentive of trying to become faster, as the player will have new options to experiment with.

Because the main theme of the game is to do with fishing, most game mechanics will reflect this. For example, instead of a normal grappling hook, the player will have a fishing rod to swing between platforms with; instead of normal projectiles being fired at enemies, all of the projectiles will be different kinds of bait to simulate catching fish in real life, like a worm or other types of lure; and instead of a normal pause menu, the buttons will also be structured to appear similarly to that of a restaurant's menu. In each of these cases, further research will be needed to correctly implement these themes.

This game will be created in the Unity game engine and coded in C# script as that is the engine the author is most familiar with. Other engines that can be used include, but are not limited to, Unreal Engine and Godot. Unreal Engine does have the ability to develop 2D games using a plugin[1], but it is primarily designed for 3D game development. Additionally, it is a lot more intensive than most other game engines. Godot has a lot of features that are very useful for 2D development, such as a pixel-based unit system and ease of use for tile maps[2]; however, it is a very unfamiliar system that the author has no experience in and so it would be significantly easier to use a game engine that is familiar, like Unity.

2.2 Objectives

The main objectives of Hook Line and Slinger are the following:

- The completion of an initial level of the game that contains a boss enemy before an end point and should take three or more minutes to complete, or less if the proper abilities are acquired,
- The implementation of a movement system for the player character, including running, jumping and its variable versions, and a grappling hook mechanic that receives positive feedback from a test audience,
- The creation of one boss enemy and at least two normal enemies, each with distinct mechanics that all attempt to damage and defeat the player,
- The addition of assets to each object in the world, with each entity containing at least one animation,
- A UI system containing a main menu, pause menu, and health bars that are designed around a fishing or sushi theme that function within the main game loop,
- And the implementation of a consistent 2D retro pixel-art aesthetic, similar to games like Mega Man and Celeste as examples of inspiration.

Each of these objectives will have several success criteria which will need to be completed for the objectives to be successful. Some of the success criteria for the movement system are as follows:

- The player can freely move left and right at a constant speed,
- The player can jump off the floor or walls an infinite amount of times,
- The player can perform a jump midair one time which resets upon touching the ground,
- The player can attach themselves to a specific point and perform an arcing motion around the point.

All objectives will have a selection of success criteria similar to this that are a deconstruction of the main point into a selection of more basic features that should each be part of the final project. Not all of these criteria are necessary for the completion of their respective objectives, but as many should be satisfied as is feasible.

2.3 Deliverables

The main deliverables of the project, excluding this report, are as follows: A fully playable first level and main menu as part of an executable file; All source code categorised and available via the public GitHub repository, which contains an informative readme.md file; an academic poster illustrating the main features of the project for investors and others interested; and a 5-minute demo video containing a run-through of the game and a highlight of the in-depth and high-level features of the game. All these deliverables are essential and needed for success.

More documentation deliverables will be inserted into this report. This includes but is not limited to a risk assessment table, flow charts for enemy AI, additional code screenshots, level designs, additional UML diagrams, sprint plans, and a kanban board. These deliverables are not necessarily needed for success but will improve the overall quality of both this report and the other main deliverables.

3 Market Research

As the gaming market is exceptionally large, the idea of a grappling-hook-based system or any other features in Hook Line and Slinger is not new. It is for that reason that market research is needed to compare this project with other video games in the market today and discern their strengths, their weaknesses, and how Hook Line and Slinger can be unique and stand out from the rest. In this section, the following games will be discussed: Hollow Knight: Silksong[3], Rusted Moss[4], Flinthook[5], Cuphead[6], The King's Bird[7], and Mega Man 11[8].

3.1 Hollow Knight: Silksong

Hollow Knight: Silksong is a Metroidvania where users play as Hornet and explore a vast world filled with challenging enemies and difficult platformer gameplay. Though the world of Silksong is technically open world, the player does not have access to everywhere immediately. Instead, the player must explore certain areas of the world to gain new movement abilities to progress with the previously unexplored areas.

Considering that this is a main feature of Metroidvanias, this project will not use the same type of exploration, but something similar instead. The mechanic of using certain abilities to progress to new areas is a nice idea, but only really works with level design that is not linear. Instead of movement mechanics creating more areas for the player to explore, Hook Line and Slinger will have different movement mechanics that provide ways for the player to take shortcuts in other levels to give players an incentive to replay levels they had previously beaten. For example, there may be a type of bait in level two which allows players to cross a gap in level one that they previously had to go down a longer path to get to the same spot.

In Hollow Knight: Silksong, there are forty-seven different bosses, including repeated fights, and some of which either test certain mechanics or become significantly easier to defeat if certain mechanics are used. For example, the Fourth Chorus, a boss in act one of the game, has a large swiping attack that is normally difficult to avoid. However, if the player uses the Drifter's Cloak that can be obtained from the area just before the boss, the player can float directly above the swipe attacks, making them significantly easier to avoid.

As another mechanic in Silksong, players gain access to a movement ability in act two known as the Clawline. When used, it launches a needle forward in a horizontal line from Hornet, before pulling her towards the stopping position of the needle. When the needle hits certain circular hoops, Hornet will be placed onto the hoop, and players can remain suspended until deciding to jump off. Some of these hoops also act as levers to open doors or activate mechanisms if Hornet stays on the hoop for a long enough time. While the use of hoops as mechanisms might not be within the scope of the project, having them as locations to swing from and even as points to hang from could be a promising idea.

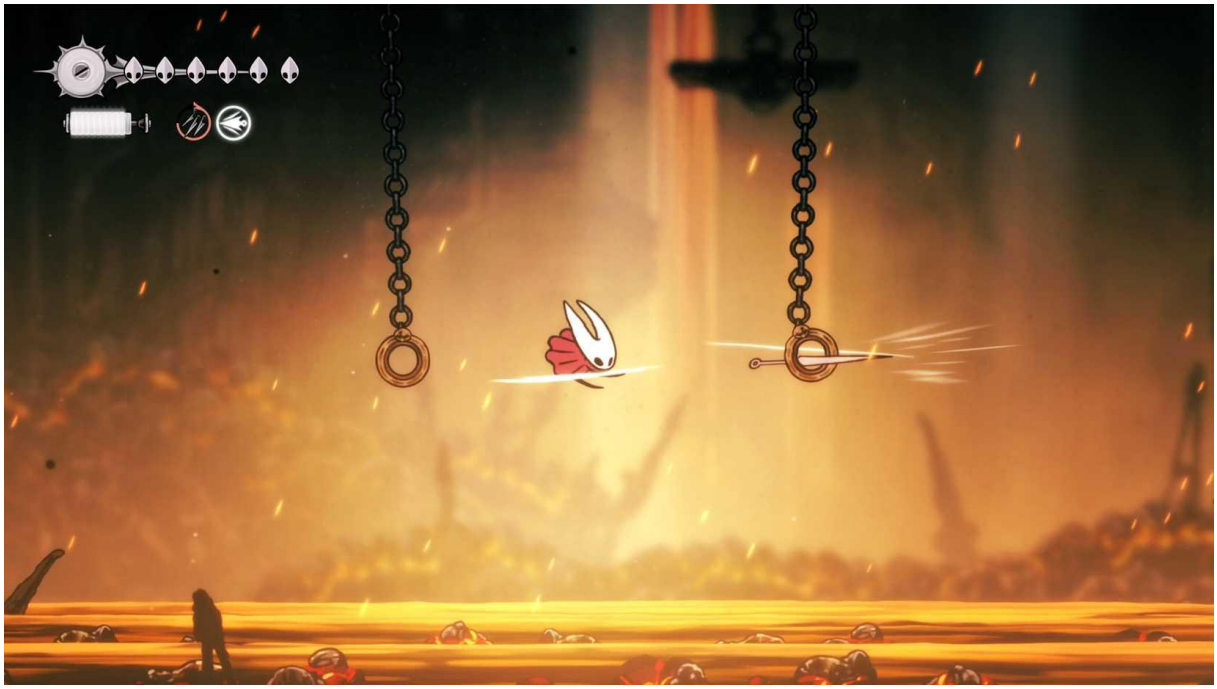


Figure I: Using the Clawline in Hollow Knight: Silksong[9]

3.2 Rusted Moss

Rusted Moss is a twin-stick shooter Metroidvania where the player moves around the world and aims in the direction of the enemies they want to shoot and the walls and ceilings they want to grapple to. The grappling hook system works similarly to that of Hook Line and Slinger's, except the rope in Rusted Moss has a more elastic feel to it, allowing players to perform actions like that of a slingshot in some instances. Because the grappling hook in this project is mostly used for more extreme motions and for larger distances and because the wire that is used in fishing rods is significantly more rigid than a normal rope, it would be better for Hook Line and Slinger's grappling hook to have a different feel to that of Rusted Moss'.

Additionally, as Rusted Moss is a Metroidvania, the player can obtain a rocket launcher as source of added movement but ends up becoming a replacement to the grappling hook in certain platforming instances. If any other movement abilities are added to Hook Line and Slinger, they cannot be a replacement for the grappling hook. Instead, they must either change how the grappling hook functions or they should function entirely differently to any other movement mechanics.

Figure II: Using the grapple in Rusted Moss[4]

3.3 Flinthook

In Flinthook, the player plays as a character that fights a series of enemies within an enclosed room using a pistol and hookshot. The hookshot can be fired in whichever way the cursor is pointing and connects to the nearest hook point in that direction before flinging the player in the direction of that hook point. While the mechanic of the character being pulled directly towards the hook point works for smaller spaces and faster paced combat, we will be sticking with the arc produced from swinging normally, as a fishing rod would not be able to pull anyone as quick as the hookshot.

However, the mechanic of pulling the character closer to the centre of the grapple point could be a good mechanic to include, considering that fishing rods can reel in the end and make the line shorter.

Figure III: Using the hookshot in Flinthook[5]

3.4 Cuphead

Cuphead is a run and gun action game that focusses on difficult boss battles. During each boss fight and other levels, the player character, Cuphead, has a variety of distinct types of projectiles that vary in terms of firing patterns and damage. For example, the default projectile, the peashooter, is a standard single projectile that fires in a straight line and does 4 damage; the chaser is a slow-moving projectile that homes in on the nearest enemy and does 2.85 damage, and the lobber is a larger projectile that arcs up before falling down to the bottom of the screen and does 11.6 damage.

While some of these abilities might be a little too strong for Hook Line and Slinger, as this project allows the player to aim due to its verticality, the idea of having various projectile types that have different firing patterns and do damage depending on how easy it is to hit enemies is something that could be used for this project well, or at least for the projectiles that are designed to do damage.

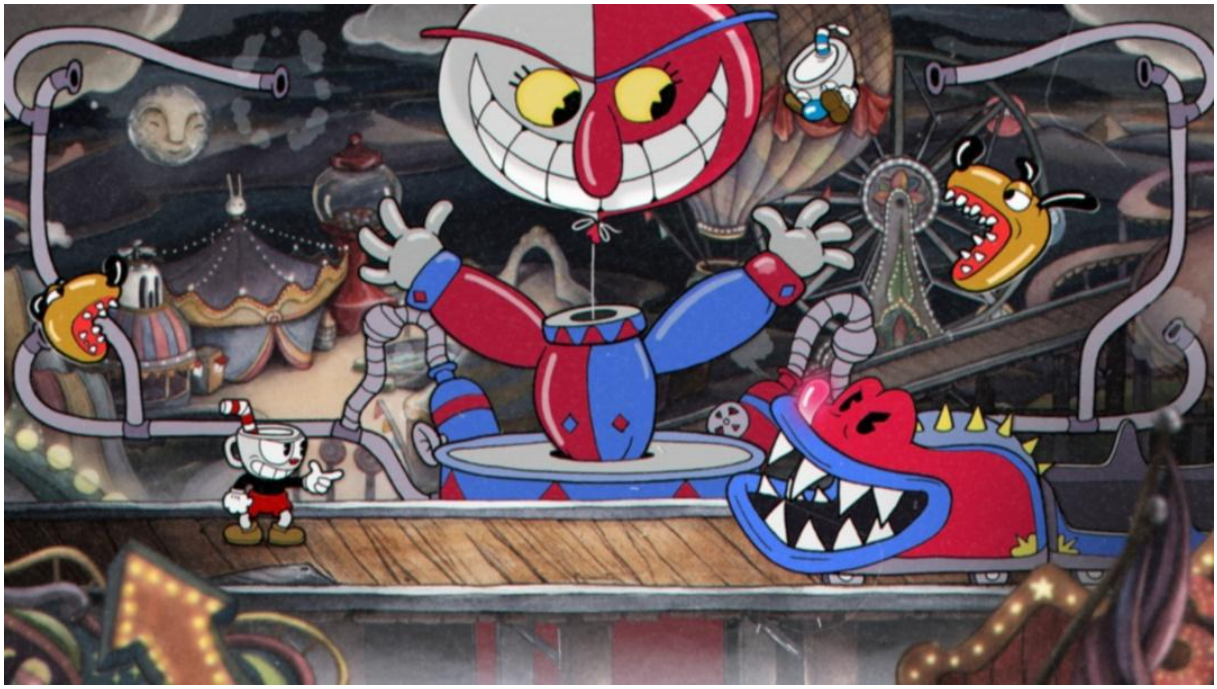


Figure IV: The Cuphead boss, Beppi the Clown[6]

3.5 The King's Bird

In the King's Bird, each level is designed with a prominent level of verticality in mind, and the player constantly alternates between high and low sections of the levels. This is because when the player falls, the character builds up momentum and allows them to fly higher than their starting position. In Cuphead's "run and gun" levels, most of the level has little verticality, except from some specific jumping challenges that complement the character's movement mechanics of a jump and a midair dash. In both instances, the movement is designed around what the player is capable of and how often it is for the player to use each of the movement mechanics.

As the main movement mechanic in Hook Line and Slinger is its grappling hook, most of the level design will focus on how to make the grappling hook enjoyable for the player. This will be done by creating large amount of verticality in its platforming sections. Like The King's bird, to maximise verticality and ease of use, the player will gain speed upon exiting the grapple after reaching a certain speed from falling. However, as there will be enemies to fight along the level, there will need to be more flat sections to contain enemies to fight; for these sections, Cuphead will be a good example to learn from, where the player can choose to fight enemies and use the more basic movement mechanics, like double jumping, to avoid attacks or they can run past and proceed to the next movement section.

Figure V: Traveling through a level in The King's Bird[7]

3.6 Mega Man 11

In Mega Man 11, each of the main levels of the game before the final Dr. Wily levels can be completed in any order, allowing the player to utilise any upgrades they obtain in one level to make their playthrough of other levels easier. For example, Torch Man is weak to the ability gained from defeating Tundra Man, who is weak to the ability gained from defeating Fuse Man.

Hollow Knight: Silksong provides a series of sections that are designed to be completed in a certain order, decided by the order in which the player obtains upgrades. Mega Man provides a series of levels that can be completed in any order the player likes, but there is technically an optimal order to complete the levels in, to make certain levels easier from upgrades in previously completed levels. Giving the player a specific obstacle that is not easily surmountable with the players current set of movement abilities, like in Silksong, is good because it gives player the freedom to explore the world but also subtly informs the player if they can go a certain way that moment or not. Like in the Mega Man Games, giving the player the freedom to choose which levels to play in which order is good and provides players with the ability to test weapons against different foes to see what works well in each area.

Both concepts will be used for the creation of this project's levels. We will give the player the ability to choose which levels to play through first and during each level, there will be one or two sections that are difficult, if not impossible, to complete with the base movement to inform the player to come back later and then give the player the necessary bait in a different level so when they return to the first level, they can

use the newly acquired bait to clear the section. In the meantime, there will be an alternative and slower path next to that section so that each of the levels can be completed. Considering that some bait is given to the player near the beginning of the level, players should only be given the opportunity to take that bait into other levels if they successfully beat the level they obtain it in first.

Figure VI: The stage select screen in Mega Man 11[10]

4 Legal, Social, Ethical, and Professional Issues

4.1 Legal Issues

As this game will eventually be published on various online storefronts, the game will need to adhere to the content policies and guidelines on those stores. For example, any content published on Steam (the largest online store for video games for computers[11]) must comply with the Steam Distribution Agreement, the Steam Subscriber Agreement, Steam Online Conduct guidelines and the Steam Privacy Policy[12]. Additionally, while not necessary, it would be ideal to obtain a PEGI rating before publishing to establish the legal minimum age for players of the game.

For video games and other entertainment, one thing to be wary of is the use of intellectual properties and other copyrighted items without permission. To avoid this, any music, art, or any other parts of my project that were not created for this project will only be used once permission has been granted and credit has been given to the original creators in the necessary outlets. Additionally, a licence is required to use Unity and to publish games with it as the game engine credited. This licence is free until the game obtains over $200,000 of revenue or funds in a span of 12 months[13].

Hook Line and Slinger will not be using the personal information of its users, only in-game data, such as the levels cleared and the player's respawn position. Therefore, the game will not need to adhere to GDPR or any other data privacy regulations, as there is no personal data to manage within the game. Any data that may be used for malicious purposes, such as account information, is all handled by the stores Hook Line and Slinger is published on.

4.2 Social Issues

For the purposes of accessibility to users of the game, all visual assets created for this project will have a clear black outline around them so users can easily discern where each object is in the game world. Additionally, colour blindness and text options are currently out of scope, but would be good features to implement in the future before a full commercial release.

As this fictional game contains violence towards animals, any violence presented in the game will be in a pixel art aesthetic to seem significantly less graphic. This is so the visuals are disconnected from what the real world looks like and players, especially children, are less likely to use it as a representation of the real world. Additionally, a disclaimer will be placed on the screen when the game is first opened to urge players to avoid harming sea life in the real world. In addition to this warning, there will be an epilepsy warning for players that suffer with epilepsy, as there are a small number of flashing lights in the game, like with grapple points and the mines that the first boss spawns.

4.3 Ethical Issues

As the demographic of Hook Line and Slinger is for all ages, the game will have to comply with the laws and regulations of rating systems like PEGI and the USK rating system in Germany among others so that the game is able to be distributed to all members of those countries that the requirements are for. For example, the fisherman originally had a design in mind that included a pipe with tobacco smoke being produced. This would call for a PEGI rating of at least a 16[14]. Therefore, the depiction of tobacco use was removed and replaced with the production of bubbles from the end of the pipe.

Considering that this project will be using various assets from other people, there is option for AI to be used to create some of the assets. However, because AI art uses other pieces of art as training data to generate new art, the game might infringe upon intellectual properties without permission. Therefore, any art and other assets that are not taken elsewhere will be created for the completion of this project.

As a single player game that is designed to be a one-time purchase, Hook Line and Slinger will not have any in-game purchases or other instances of monetisation outside of the initial purchase of the game, which will be free initially. If the game is continued after this project is over, then it will receive a price tag indicative of its quality that is also competitively priced for the genre in mind.

4.4 Professional Issues

Once the game has fully been released to the public to play, professionalism will need to be considered when marketing, communicating, and updating the game. To avoid any chance of false advertising, any trailers or other pieces of advertisement will need to be as accurate as possible when describing the features and gameplay of Hook Line and Slinger. The advertisement will avoid any profanity or other features that are against the terms of service for any of the sites it may appear on and it will accurately reflect the age rating of the official game.

All changes to the game will be accurately recorded on the GitHub repository to successfully communicate with potential investors and future employees. Substantial changes or collections of minor changes will be released to the public in the form of patches alongside a list of all changes that are correctly communicated to form a bond with the community of players and to show integrity with everything communicated to those that wish to know.

The game will feature no game-breaking bugs or broken features that were found during testing before the game's full release and any other bugs found by players that are reported will be fixed within short notice.

5 Method of Approach

5.1 Development Methodology

This project will utilise the Agile Software Development Life Cycle (SDLC) for this year, in the format of 2-week sprints. Every two weeks, a meeting will take place with this project's supervisor to discuss what happened in the previous sprint, what went well and what went wrong, and what will be done in the next sprint. Afterwards, each feature to be completed in the sprint will go through the SDLC to ensure its completion. These steps in the cycle are as follows:

i. Analysis: is the feature part of the objectives listed in 2.2 and what need to be done to complete or update this feature?
ii. Design: how can feature be implemented into the project and how will it look or feel for the player?
iii. Implementation: has the feature been correctly implemented into the project?
iv. Testing: does the feature work effectively and does it successfully perform the task outlined in the objectives?
v. And maintenance: does this feature require further development or other forms of maintenance in the future?

Once a feature has gone through the SDLC, if needs further maintenance, it can go through the SDLC an additional time to fix or fully develop that system.

This is the main reason the agile ideology is typically better over other versions of the SDLC, such as the waterfall methodology. The waterfall method of SDLC is like agile, except the cycle is only gone through once and each step cannot be started unless each of the previous steps have been completed. This is good for significantly smaller projects but for Hook Line and Slinger, each feature is part of a larger system and requires multiple iterations to be fully realised. Waterfall means that a lot of features would not be as well designed or implemented as if they were done so with agile.

5.2 System Architecture

The game in Unity will be split into several scenes, each level and the main menu each being a scene, and each scene will contain all game objects that are designed to be there permanently within its hierarchy. And additional game objects that are not immediately within the hierarchy will be saved as prefabricated objects and created from whichever piece of code requires them to exist, such as projectiles being spawned by the player. Each scene can be accessed from the main menu and vice versa.

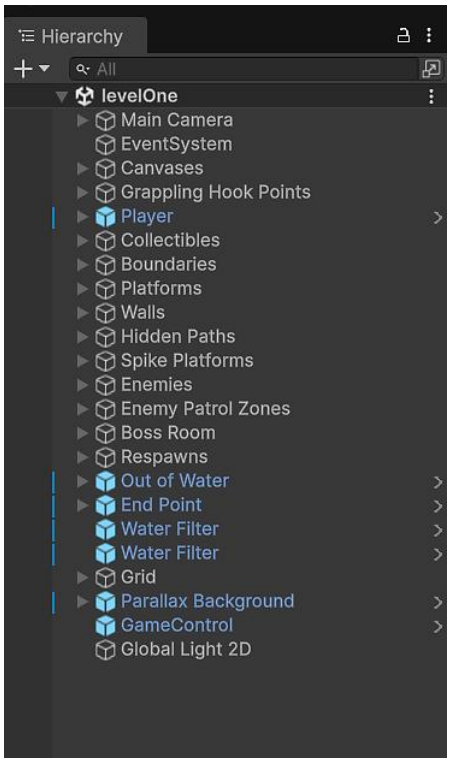


Figure VII: Level one's object hierarchy

Each non-UI game object will have a transform component to determine is size and position within the world, and then any other necessary components can be added as needed, such as a collider or a sprite component. Additionally, some game objects will have C# scripts added as components to dictate how the object and its components interact with the world.

Each of these scripts that are mono-behavioural contain a Start and an Update function by default and the Update functions for the scripts will be our games main loop, as they are called once every frame. Other scripts that determine other parts of the game, such as gravity and lighting, also apply every frame and so they will also be part of our game loop.

Within the main game loop, the game should manage movement and combat for the player character, enemy AI, timers and cooldowns, animations, the detection system for grapple points, and other objects with triggers and other states that need to be constantly checked. Any other functions that the C# scripts contain are called from inputs or Update.

In addition to other C# scripts that use the Update function, some scripts inherit the functionality from other scripts and uses them in new ways. For example, as shown by some of the code in the appendix, the projectile script has the main functionality that determines damage and its speed is determined in the Start function. The worm builds on this by inheriting the functions in projectile and then uses the Update function to change its upwards velocity in the form of a sine wave, mimicking the action a worm would make on the ground. This is one of a few different scripts that inherit from projectile, as shown in the class diagram.

5.3 Game Design

The main movement of the game is simple to code. If the player inputs left and right, they move in those directions and pressing jump launches the player a certain distance into the air. The player will have one extra jump they can perform midair until they touch the ground again. If the player is against a wall while midair, they will slide down it at a slower speed than the normal falling speed and it can be jumped off at any point.

When the player is close to a grapple point, they can attach themselves to it and swing around an area or obstacle and build up speed to be flung across a room. While grappling, the player needs a line of sight to the grapple point to avoid players getting stuck in certain part of the level. This way, the player can stop grappling by letting go of the button or by getting cut off by the corner of a pillar. If the player holds the jump button while grappling, they will be reeled in towards the centre of the grapple point; this can be used to build up speed while grappling or to reach areas above without needing to wall jump a lot. This is shown in the flow chart below.

Additionally, the player is rotated around their grapple point depending on their position, rotation, and current animation frame, as shown in the code samples below. Seeing as there are three main frames for the swinging animation (the fishing rod being directly in front of the player, directly above, and diagonally between the two), the grappling determines the angle the player needs to be at so the rope between the end of the fishing rod and the centre of the grapple point looks natural.

While moving around, the game will take the player's cursor and obtain the angle between the player character and cursor and move a game object called the fire point to the same angle so that when the player fires a projectile, it will go in the direction of the cursor; this will be the way that players can aim and fire at the enemies in each level.

In each level, the game will have various enemies with different attack patterns; however, each of them will have the same initial AI. Each enemy will patrol their specific area and attack upon detecting, or being damaged by, the player. The more common enemies will attempt to get within a certain range and fire projectiles in the direction of the player. This basic pattern has been designed with this state machine as its initial design.

The main designs for creating levels will follow the principle of utilising fluctuating height levels to provide a sense of verticality for the player. There will be some larger flat surfaces throughout that will contain enemies before placing a large boss arena at the end of the level for the player to be tested against a formidable foe.

When the player reaches the end of the first level, they will face a dastardly foe known as Minehead. Minehead is a squid trapped inside a mine and wants revenge against his prison's architects. The main boss stage will be split into three sections, each of which being a platform higher up in the stage, with its zenith being the third and final stage. During each stage, the boss will teleport to a central point in that phase area and spawn two mines, tasked with following and damaging the player, before teleporting to another place in that stage. This will be repeated until Minehead moves to the next location or is defeated.

5.4 Tools and Technologies

The main and only engine that Hook Line and Slinger will be created in is Unity, with all the scripts written in C#. As of the 23rd of April 2020, Unity has implemented an input system which predefines the inputs for various devices and can each be assigned to call different functions[15].

This is a particularly useful system that saves a lot of time with writing the input functions and results in a lack of exceedingly long if statement names. It also makes things significantly easier for adding new kinds of device as inputs. As shown in the appendices below, there is the initial set up of each input and respective function, but the initial input readings are a lot less cluttered.

Another system that Unity has are joints. Joints allow game objects to connect with another object and force them both to be physically affected in a certain way, depending on the type of joint. For example, a hinge joint acts like the hinge of a door that automatically shuts after being pushed.

All the art that is created for this project will be 64-by-64-pixel art that is created on a website called Piskel[16]. Piskel can have multiple frames and preview the full animation before exporting, and then each frame can be placed in a single spritesheet, which makes things quite easy when exported into Unity.

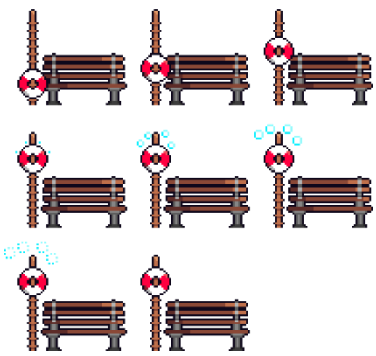


Figure IX: Animation spritesheet for the respawn point

Figure VIII: Animation spritesheet for the fisherman's idle position

5.5 Project Planning

As shown in the Gantt chart, the project is designed with 2-week sprints in mind and will implement each section of the game in a suitable order. As Hook Line and Slinger is focused on its movement, the first order of business will be to implement the character controller. This will be each of the core features of the movement and should be implemented within the first few sprints, as this will affect the progress of the other features. After the base movement and grappling system have been completed, the next priority will be the combat systems. This should take as many sprints as the movement to implement the firing of projectiles, basic enemy objects, and health and damage to finalise the basic combat.

Once most of the initial gameplay has been completed, the next step is to make the game look nice. Models and animations for the character, enemies, bosses, and the projectiles are required, as well as the textures used for the platforms and backgrounds. While some of this can be found online and obtained with licensing, there are other assets which will be more difficult to find: therefore, these assets and animations will need to be manually created, which will require several sprints to create, due to a lack of experience with the creation of art.

Once the animations for enemies and bosses have been created, it will become significantly easier to design the mechanics and AI for those entities. This, alongside various fixes and improvements, will be the last large section of sprint before the end of the project, taking up multiple sprints to finish the first level and its constituents. After the finalisation of level one, there will be one sprint to implement sound and music, and another to implement the menus, as these are both systems that require most other features to be completed and can independently be completed in a short amount of time.

5.6 Risk Management

As shown in the Risk Assessment Table, there are many risks to consider when creating a project for commercial use; however, all these risks can be mitigated to varying degrees. The biggest risk that comes from this project is an abundance of bugs which can appear from various sources. This is a highly likely risk to happen and can cause many issues, depending on the bug, so it is a significant risk for the project. The biggest mitigation for this risk is by testing the game throughout the life cycle of the game; this way, any bugs can be spotted around the time that they are implemented and any others that slip through the cracks will be found in later stages with user testing. Additionally, as part of the professionalism that comes with the project, support will be available after full release to fix any bugs that are reported by players.

Other risks include poor time management and a loss or destruction of the project's device; both result in the inability to submit a working game. With poor time management, this can easily be mitigated with agile methodology and sprint meetings to ensure that the project does what it sets out to do every two weeks. In the unlikely event that the original device making the project is unavailable, the project will be stored in an online repository, GitHub, so that it is accessible on any device. Almost all other risks that could appear in the project are covered by what is outlined in LSEP.

6 End-project Report

6.1 Evaluation of Objectives

The first and most essential objective for the beginning of the project, as outlined in 2.2, is to create the character's movement with a fun feel. If this objective is not completed, this project has overall failed. The main success criteria outlined to complete this objective is the implementation of basic character movement, such as moving left and right and jumping, and of the grappling hook mechanics. The character movement has been fully achieved; the player character can move around the scene and jump while on the ground and once in the air. If the player is next to a wall, they will slide down it after an amount of time and the player can jump off it at any point.

The grapple point is fully completed but does not entirely work as desired. When the player is within range, it checks if there is a wall in between the point and the player. If there is not, the player attaches to that point. This is functioning as intended; however, the player cannot accurately swing between points that have overlapping radii. In hindsight, the script to determine grapple-able points should have been placed within the player movement script instead of the grapple point script.

These two sections of the movement mechanics together show that the objective of implementing the movement system is only partially complete, due to the grappling system not fully functioning as intended. As part of this, the feedback on the movement was positive, as shown in the analysis on testing.

The next main objective is a selection/variety of bosses and enemies. For a first level, the success criterion of making a boss enemy with multiple stages was successfully completed. The boss starts at the bottom of a tall room, and with each phase, the boss climbs higher in the room until the player defeats the boss at the top. The success criterion of creating distinct types of enemies that try to defeat the player was only partially completed, as only one enemy was created instead of a plethora. This was mostly due to time constraints, but the main structure of the enemy AI was completed for future versions of enemies. The altering of damage numbers also made combat function and feel better for players over the original stats.

Another big objective for the creation of this project is the creation of assets and animations for the objects in the game, especially considering that the mechanics for the boss are directly tied to its animations. Initially, the background and terrain were the first assets obtained until the rest were made using Piskel. Once the animations for the in-game entities were finished, this objective was fully achieved. Additionally, the pixel art assets created for this project were designed in such a way that it also satisfies the objective of creating an aesthetic like that of older 2D platformers. This objective was fully completed, although some assets will need to be changed when releasing commercially.

Finally, as another objective, this project must contain a user interface that is indicative of a game related to fishing, which encompasses the main menu, pause menu, and any other visual elements that are not within the actual game world. This has overall been mostly successful. The pause menu is designed to like a menu found at a sushi restaurant, with a basic wallpaper on one side and the different menu options with prices on the other. For the main UI of the game, the player health bar contains a heart with an anchor wrapped around it and the boss health bar has waves on either side, both of which help simulate a feeling of the player being underwater. The main menu did not fully achieve the feel of a restaurant, as the idea behind the main menu was to be the player's restaurant and then they go out into levels to catch fish. However, there were no easily accessible assets online that were similar enough to the inside of a restaurant, so the background on the main menu is instead an image of a dock. Overall, this objective has been fully completed.

6.2 Changes During the Project

During the initial stages of the project, the game started off as a Unity project before moving over to Godot to experiment with which engine would be more suitable for implementing a grappling hook; however, it was not long before the project was moved back to Unity, due to a lack of understanding of Godot's functionality and engine-unique script. This did not have a significant impact on the project but did delay some of the implementation by a sprint.

The biggest and most impactful change that was made to this project is the save system. From the initial implementation of the save system, the plan was to save data with a class called PlayerPrefs. PlayerPrefs was useful for the initial saving of variables like the player's respawn position but stores data differently depending on the device and operating system the game is running on. Additionally, after implementing the system for different projectiles, it was discovered that PlayerPrefs was not ideal for saving data, as it can only store floats, integers, and strings.

Because there is a list of different projectiles available to the player, the save system has been changed to a dat file as shown in the appendices, so the game can save arrays of projectile indexes and enumerators for which input device the player is using. However, due to an oversight, the respawn coordinates were not simplified down to a singular 2D vector and were instead kept as two separate float values.

As shown in the appendices below, the save system is designed to keep data like the aforementioned respawn coordinates stored so that when the scene is loaded, either from entering the level from the title screen or by respawning and reloading the scene, the game will take the coordinates stored in the game control and move the player to the stored position.

Initially, the damage used smaller numbers for damaging enemies and players, with the default projectile dealing 1 damage and enemies have 5 health on average. Because of bait that was designed to do less damage than the default, all original damage and health numbers were multiplied by 10. This made things significantly easier to adjust any necessary numbers and appeared nicer to calculate. From this point onwards, there were various times where numbers would need to be adjusted to make the game more fair or difficult for the players.

For the hidden paths in the game, they move in the direction the player is moving while grappling. However, the paths used a code similar to the grapple points, except with the ability for the player to freely move in the air, which didn't look as physically accurate as it should be, so the code was changed so that the character can only open up the hidden path while on the ground. This overall looked a lot better. While this did not affect the project in a way that would result in the changing of other code, this change would result in confusion from the tutorial message for grappling. The original message for the grapple tutorial stated that the player can only grapple while midair; now the tutorial message states that the player can grapple to any point that is glowing.

6.3 Testing

The biggest instance of testing performed was at the usability testing session on the 25th of March 2026. During this session, students go around and try out each other's projects and give feedback. By the end of the testing session, twelve students tested the game, and each piece of feedback was directly integrated into Hook Line and Slinger. For each question, the responses can be found below and will be discussed in detail here.

6.3.1 Level Completion Rate

The first major question given to the players was whether they were able to fully complete the first level, which ended up being a fifty percent split between yes and no. From those that did not complete level 1, some were because the players decided to stop playing after being satisfied with what they played, some gave up after being unable to defeat the boss, and a couple of players were unable to get past the first jump. This is roughly the desired outcome, as the game would be too easy if every person who tested the game could complete the level in the given time frame; however, the instances where players could not complete the first jump most likely show a disconnect between less experienced players and the basic mechanics.

6.3.2 Player Previous Experience

Regarding the players' skill levels, all players stated they had experience with video games, but two of the twelve players did not have any experience with 2D platformers. Surprisingly, only one of the two players that failed the first jump was inexperienced with 2D platformers. This would imply that regardless of the player's skill level, there will be some players that are unable to properly utilise the game's main mechanics.

6.3.3 Alternate Bait Usage

The next question given to the players was whether they used the alternate bait within the level, what they thought about them if so, and why they did not use them if not. Of the twelve players, six of them did not try out the alternate bait, with three of them not knowing how to and the other three not wanting to. For those that did not know how to access other bait, the icon at the respawn point has been made more noticeable. A tutorial for the projectile menu has additionally been placed next to the collectable so the player knows what to do when they get a new type of bait.

If a player does not want to use a type of bait, they should not be forced into using it; however, if a large enough number of people choose to avoid a specific bait, or if enough players dislike using it, then that bait in particular will need to be reworked in some way and any feedback on it will be taken into consideration.

6.3.4 Likert Scale Questions

For the next part of the testing questionnaire, the players were given a series of statements that they decide whether they agree with them or not. The only statement of note is “The tutorial pop-ups were helpful in teaching me the controls,” as none of the other statements had any disagreements. The main reason that some players disagreed with this statement is because at the time, the tutorials only showed the controls for keyboard and mouse, even when the player was using a controller. It has since been changed so the game checks which input device is being used and changes the text to show the necessary controls.

6.3.5 Difficulty Rating

In terms of the combat, the players were given two main questions: how difficult were the base enemies and how difficult was the first boss, Minehead? For the base enemies, most players rated the difficulty of the base enemies at a 3 out of 5, which represents the difficulty being just right. Because the other ratings leaned more towards the enemies being easy, the enemies were changed to fire projectiles in a burst of three bubbles; this made those that found the enemies too easy more likely to be hit with their projectiles, and the players that found the enemies difficult or just right won’t be as affected as much, due to the cooldown between instances of damage for the player.

The boss fight is a lot more split on its difficulty, partially because its main attack, the mines, did not work properly for the first few participants. This meant that most of those that found the boss too easy were those that could do the platforming well and could not be damaged by the boss. After the mines were fixed, due to them not having a player movement script to damage, the number of players that found the boss difficult increased, so the mines had their damage reduced and their spawn points moved further away from the main platforms.

6.3.6 Written Feedback

Finally, there were a couple of sections for feedback and features to improve. Many improvements mentioned were specifically to do with the controller version of the game; therefore, like with the tutorials before, these changes were fixed quickly such as changing how the movement input works to function correctly when swinging on either input device. Another useful feature suggested was making the wall sliding more forgiving, which was implemented by adding a second where the player stays in place before sliding down.

Additionally, there was one bug that a couple of people found where the player would teleport next to a wall when falling in a spike pit, instead of up to a platform above. This is due to the walls around the spike pit being the same layer as the ground and the player rotating whenever they are swinging, so it detects the wall as a piece of ground and sets it to the new teleport location. This was an easy, but time-consuming fix where each wall was given a new layer and any ground above was given a thin line with the ground layer so that the player will only be grounded when standing at the very top of a piece of terrain.

As well as all these changes and feedback, every player enjoyed the grappling mechanics and thought they were fun, which is exactly what the movement was aiming to accomplish. This is shown by the feedback in the appendix.

6.4 Overall Project Success

A good amount of the project went better than initially expected, with a large amount of the bugs that appeared being fixed soon. Despite this project going well, it would not be fair to say there were no hiccoughs along the way, the main issues that arose being the initial implementation of the grappling system and the logic for the hidden paths.

As shown in the evaluation of objectives, each of the objectives listed at the beginning of the report have been completed, other than a couple. With these objective and success criteria reached, this project has become a fully-made first level of a 2D platformer as desired with a fun and well-designed grappling hook system attached, as proven by the given feedback.

7 Project Reflections

7.1 Reflection on Development Process

As shown in the Trello board, the development process stayed within the confines of the agile ideology. Each section of the game development was designed and implemented within the confines of a two-week sprint, except for a couple specific tasks that require other features to be completed beforehand, like the overall design of the first level. The order in which tasks are completed, such as movement, combat, and visuals, were roughly completed in the same order as what was outlined in the Gantt chart.

In both instances, the first part of the game was the implementation of the basic character movement, followed by initial combat mechanics, and assets and animation. The main differences between the two was the two sprints at the beginning of the project, which did not include any coding or game development, and the implementation of enemy AI and menus, which were developed earlier than originally planned.

Something that should have been implemented with better time management would be the sound and music. The main menu, first level, and boss fight should each have their own distinct music scores and certain actions should have their own sound effects. These sound effects should only be for significant action that don't happen often enough to annoy the player, like mines or the player firing projectiles, as they are on a set timer.

Both music and sound effects should be tied to sliders in the settings menu to change the volume and each slider should have the name of a fish related to that setting, to provide the feeling of a restaurant menu. As part of this, the percentage symbols usually present in sound settings are replaced with prices to simulate the fish being items that customers can order, like in the diagram below.

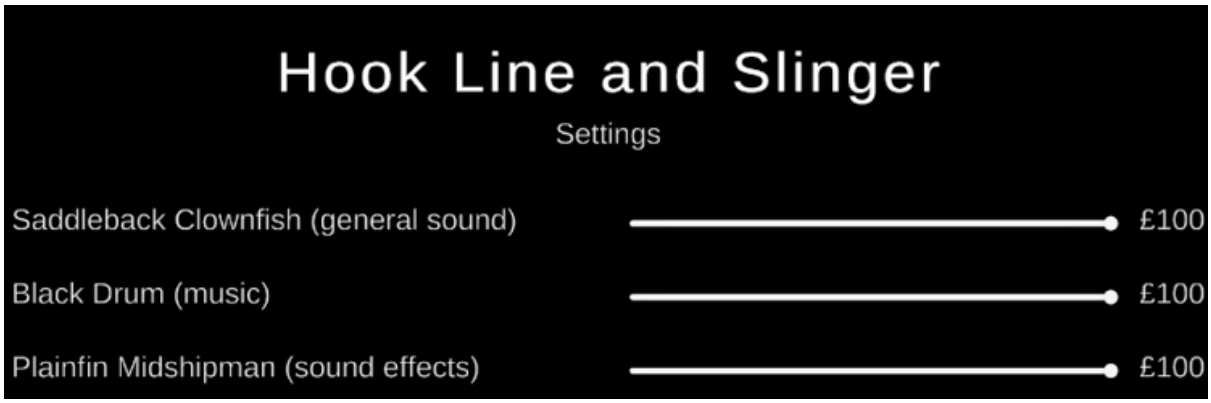


Figure X: Substitute settings menu with fish as sound settings

7.2 Reflection on Technical Decisions

In the end, the engine choice of Unity over other engines like Unreal Engine and Godot was not a truly relevant decision; however, the prior knowledge and experience of Unity helped with implementation and sped up the process significantly. The decision to use the built-in physics for Unity and have each of the rigid bodies set to be dynamic was less time consuming overall and made it easier to implement the grappling system. However, as this is advertised as a physics-based game, it would have been better to set the physics type to kinematic and code a physics system that complements the desired movement mechanics.

On the topic of the grappling system, it worked as designed but did not work for multiple grapple points in proximity. Additionally, if the physics system is to be changed to kinematic from dynamic, a joint different to the distance joint might be a better solution.

In the creation of the boss mechanics, each step of Minehead's attack patterns uses a coroutine as a way of performing animations and pauses in code asynchronously to the main game loop, such as teleporting to a different position like what is shown in the appendices. There is a minor problem with the way it is currently coded, where there are many "if" statements to check exactly what phase the boss is currently in, which does not scale well for the mechanics if more phases are added to this boss or another. If this is to be changed, it should be altered to the form of a State Machine.

7.3 Reflection on Personal Performance

Throughout the course of this project, the amount of time given to the development of Hook Line and Slinger has increased with each month or so. Ideally, a large amount of development and documentation could have been started sooner than when it realistically did start. That way, there would be more time for the implementation of sound. Additionally, if more time were put into the project throughout the year, there would have been more of an opportunity to commence unit testing during the iterative process.

An irritation that happens with coding in Unity, is that Visual Studio does not properly recognise functions and other pieces of coding that are unique to Unity; therefore, if there are any bugs or issues that appear in the code, there is no way of telling until Unity has reloaded and the error appears in the console log. This did not decrease the chance of a problem being solved promptly, only an increase to the amount of time to register the error in the first place.

7.4 Reflection vs Market Research

Using what was learned with Hollow Knight: Silksong and Mega Man 11, some of the platforming was designed to have shortcuts that could be cleared with bait in other levels, so the player could get a faster time. This desire to obtain a faster time is common among players, as shown by most responses to question 6 of the user testing.

As decided upon comparing with Flinthook and Rusted Moss, the grappling hook was designed to be taut often and the player was designed to move in a more arcing movement, rather than elastic or pulling motion. This ended up being the most suitable version of grappling for how the terrain was designed and was found to be a fun system, as shown by the feedback in question 11 of the user testing.

7.5 What Could Be Done Differently

If an understanding of how it should function was met, the rigid bodies could be changed to kinematic. This will integrate a game-specific physics engine, instead of fighting with Unity, which will allow movement to function exactly as desired.

For the grappling system, it would have been a much better idea to put the grapple detection logic into the player movement script instead of as its own script unique to the point. This way, the player can then detect the closest grapple point within a certain radius and grapple to one at a time, instead of only being able to have one grapple point in any one area. Regarding grapple points, the debris that appears from breaking a hidden wall should have been an animation, rather than physical objects that fall, as it would simplify the code for the hidden path script.

Another sizeable portion that could be changed would be the menus, especially the main menu. The pause menu could have more research put into what a typical sushi restaurant's menu looks like, and the main menu should have been more like the original design of a restaurant with customers and places for the defeated bosses. In addition to the look of the menus, some of the other assets could be changed if there was enough time, like the terrain or making normal maps for the entities to accurately reflect the light.

In the design of the first level, there are some elements, like the position and nature of the shortcuts, which could have been thought out more clearly from the get-go. Additionally, if more time were available, there would be more types of enemies that appear in the first level and the animations for the fish would be created before the AI to properly integrate its mechanics into the visuals. This would make things easier for creating new enemies in the future, as all that would need to change is the animations and the attack function.

8 Conclusion

During this project, a fully functional first level of Hook Line and Slinger has been created that meets each of the objectives outlined previously with a fun and engaging grappling hook system integrated alongside. This, alongside a more niche premise of a fisherman on an adventure, creates a game that fits into a specific spot in the market that few other games occupy.

While a couple of objectives were not fully realised to the amount of the initial project's vision, each piece of success criteria was completed to an acceptable amount, regardless of any limitations placed on the project, such as time constraints or a lack of a team.

In the future, the remaining levels will be completed, being between four and six levels collectively with all but one being other levels that can be done in any order alongside the first level, and the final level being a large boss fight that requires each other level to be completed. Additionally, any assets taken from online will be replaced with game-exclusive new pixel art to assure that each piece of art is tailored to suit its purpose within the game, such as a main menu that has the look of a sushi restaurant that contains the defeated enemies as dishes.

Another feature that can be expanded is the enemy AI. From the base implementation of enemy mechanics, the only thing that needs to change is the attack functions to align with its respective enemy designs, which can be done with the inheritance of the base fish class.

Overall, Not only has the requirements of each objective been completed, either partially or fully, it has resulted in the foundations of a game that has received nothing but positive ratings from testers, all while providing the base for a game that can expanded

9 References/Bibliography

9.1 Other game engines

9.1.1 2D in Unreal Engine

Epic Games Developer. (2026). *2D in Unreal Engine | Unreal Engine 5.7 Documentation | Epic Developer Community*. [online] Available at: https://dev.epicgames.com/documentation/unreal-engine/2d-in-unreal-engine [Accessed 20 Apr. 2026].

9.1.2 2D in Godot

Engine, G. (n.d.). *Features*. [online] Godot Engine. Available at: https://godotengine.org/features/.

9.2 Game research

9.2.1 Hollow Knight: Silksong

store.steampowered.com. (n.d.). *Hollow Knight: Silksong on Steam*. [online] Available at: https://store.steampowered.com/app/1030300/Hollow_Knight_Silksong/.

9.2.2 Rusted Moss

Steampowered.com. (2019). *Rusted Moss on Steam*. [online] Available at: https://store.steampowered.com/app/1772830/Rusted_Moss/.

9.2.3 Flinthook

Steampowered.com. (2026). *Flinthook on Steam*. [online] Available at: https://store.steampowered.com/app/401710/Flinthook/

9.2.4 Cuphead

Steampowered.com. (2017). *Cuphead on Steam*. [online] Available at: https://store.steampowered.com/app/268910/Cuphead/.

9.2.5 The King's Bird

Steampowered.com. (2026). *The King's Bird on Steam*. [online] Available at: https://store.steampowered.com/app/812550/The_Kings_Bird/ [Accessed 22 Apr. 2026].

9.2.6 Mega Man 11

store.steampowered.com. (n.d.). *Save 67% on Mega Man 11 on Steam*. [online] Available at: https://store.steampowered.com/app/742300/Mega_Man_11/.

9.2.7 Using the Clawline in Hollow Knight Silksong

Yu, J. (2025). *Hollow Knight: Silksong walkthrough*. [online] Polygon.com. Available at: https://www.polygon.com/hollow-knight-silksong-walkthrough-credits-acts/ [Accessed 16 Apr. 2026].

9.2.8 Level select in Mega Man 11

Fajardo, M. (2018). *Just Push Start*. [online] Just Push Start. Available at: https://www.justpushstart.com/blog/2018/10/01/mega-man-11-review/ [Accessed 17 Apr. 2026].

9.3 LSEP

9.3.1 Steam's player base

Wolens, J. (2025). *Steam is officially bigger than Canada with a 41-million concurrent user record*. [online] PC Gamer. Available at: https://www.pcgamer.com/software/platforms/steam-is-officially-bigger-than-canada-with-a-41-million-concurrent-user-record/.

9.3.2 Steam Partner Program

Steamworks (2019). *Steamworks Partner Program*. [online] Steamgames.com. Available at: https://partner.steamgames.com/steamdirect.

9.3.3 Unity Pricing

Unity. (2025). *Unity Plans & Pricing: Pro, Personal, Enterprise, Industry | Unity*. [online] Available at: Unity Plans and Pricing [Accessed 17 Apr. 2026].

9.3.4 PEGI ratings

PEGI (2017). *What do the labels mean? | Pegi Public Site*. [online] Pegi.info. Available at: https://pegi.info/what-do-the-labels-mean.

9.4 Tools

9.4.1 Unity Input System

Input system: Input system: 1.19.0 (no date) Input System | 1.19.0. Available at: https://docs.unity3d.com/Packages/com.unity.inputsystem@1.19/manual/index.html (Accessed: 25 April 2026).

9.4.2 Piskel

www.piskelapp.com. (n.d.). *Piskel - Free online sprite editor*. [online] Available at: https://www.piskelapp.com.

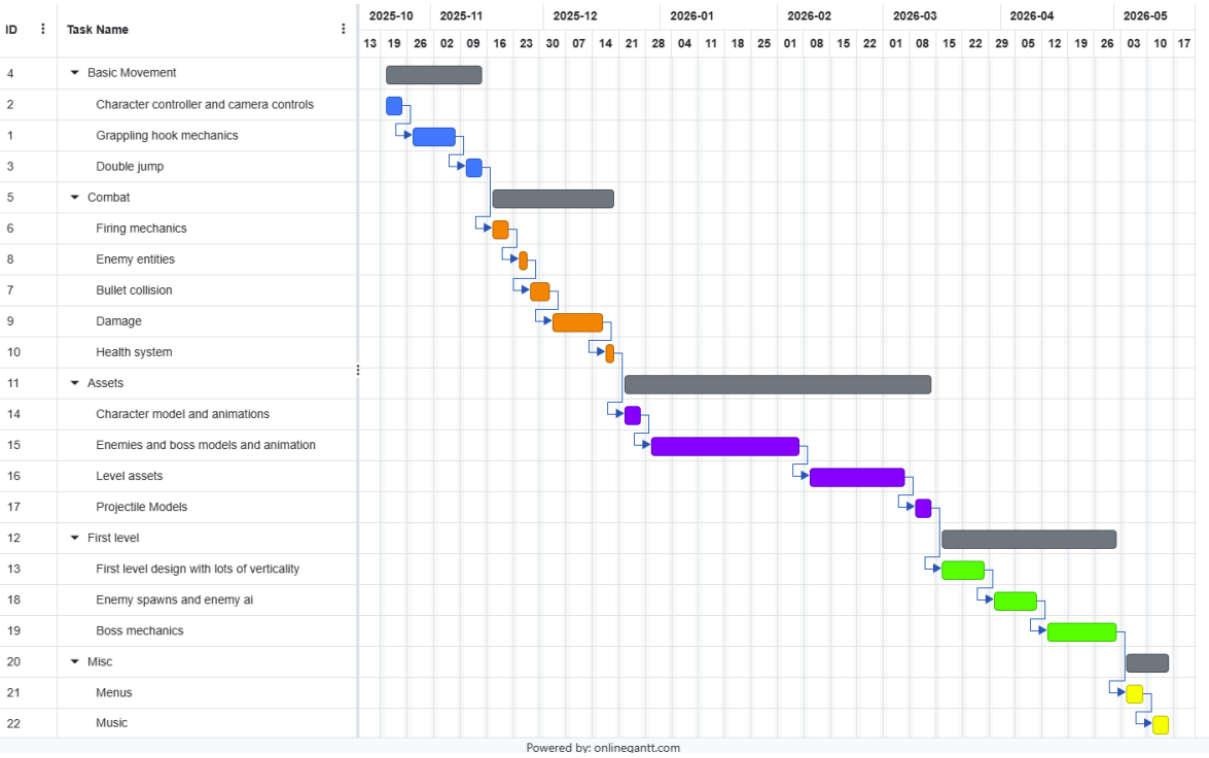
10 Appendices

10.1 User Guide

To play Hook Line and Slinger at this moment, the player needs to go onto the GitHub repository and clone it. Then running the executable file in the application folder will run the game and any data saved will be stored in the player's personal C: drive.

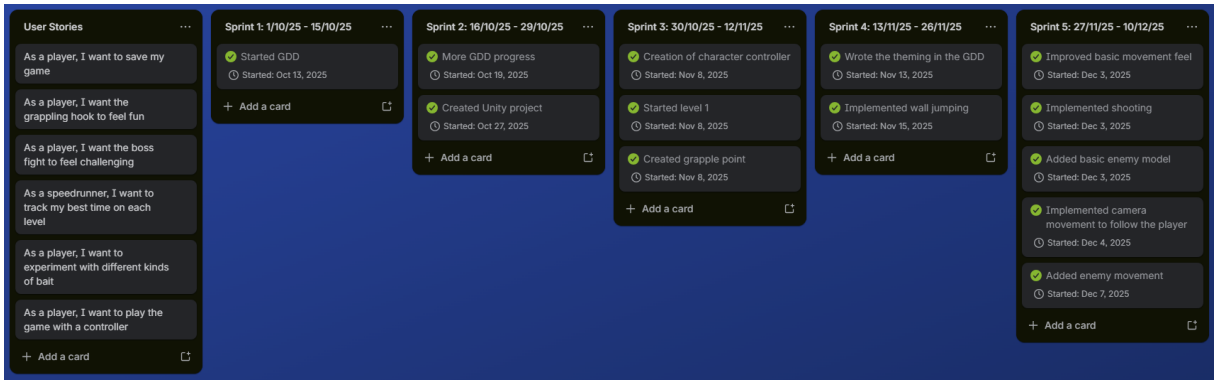
10.2 Project Planning & Agile Evidence

10.2.1 Gantt Chart



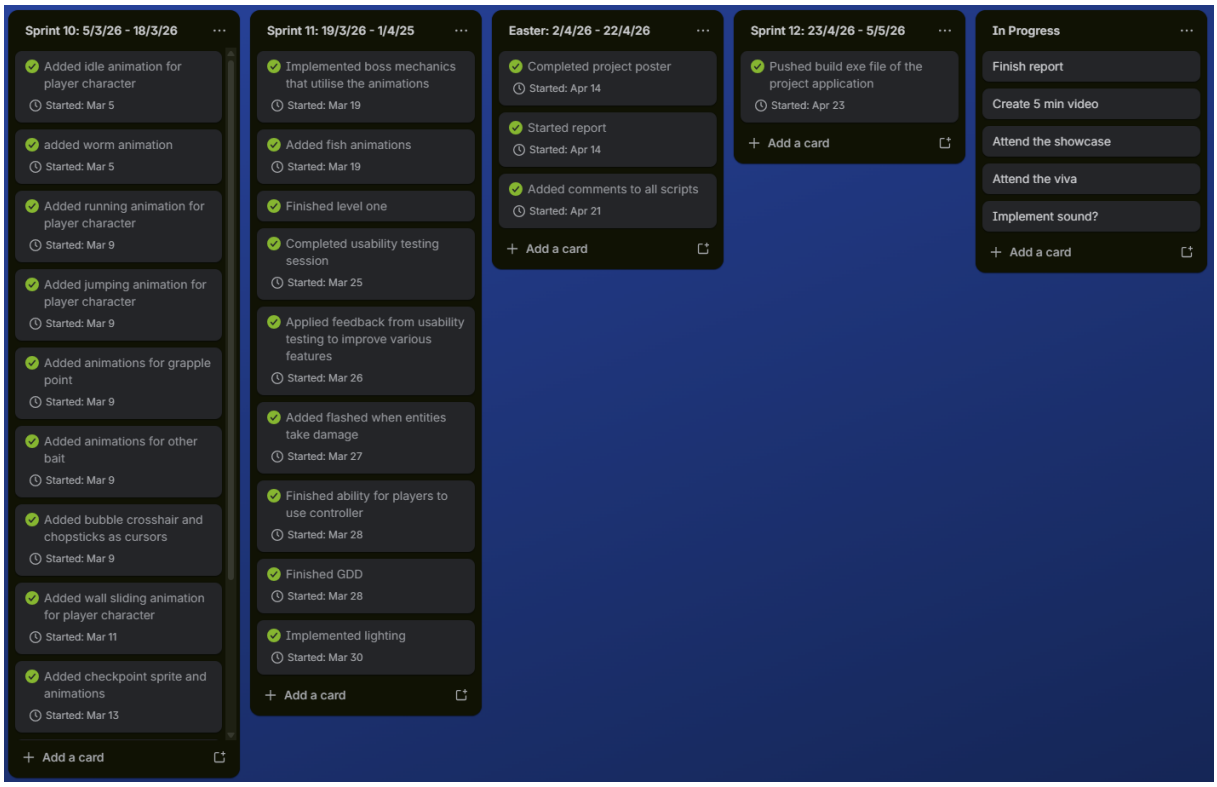
Appendix A: Initial Gantt chart

10.2.2 Trello board with all sprints



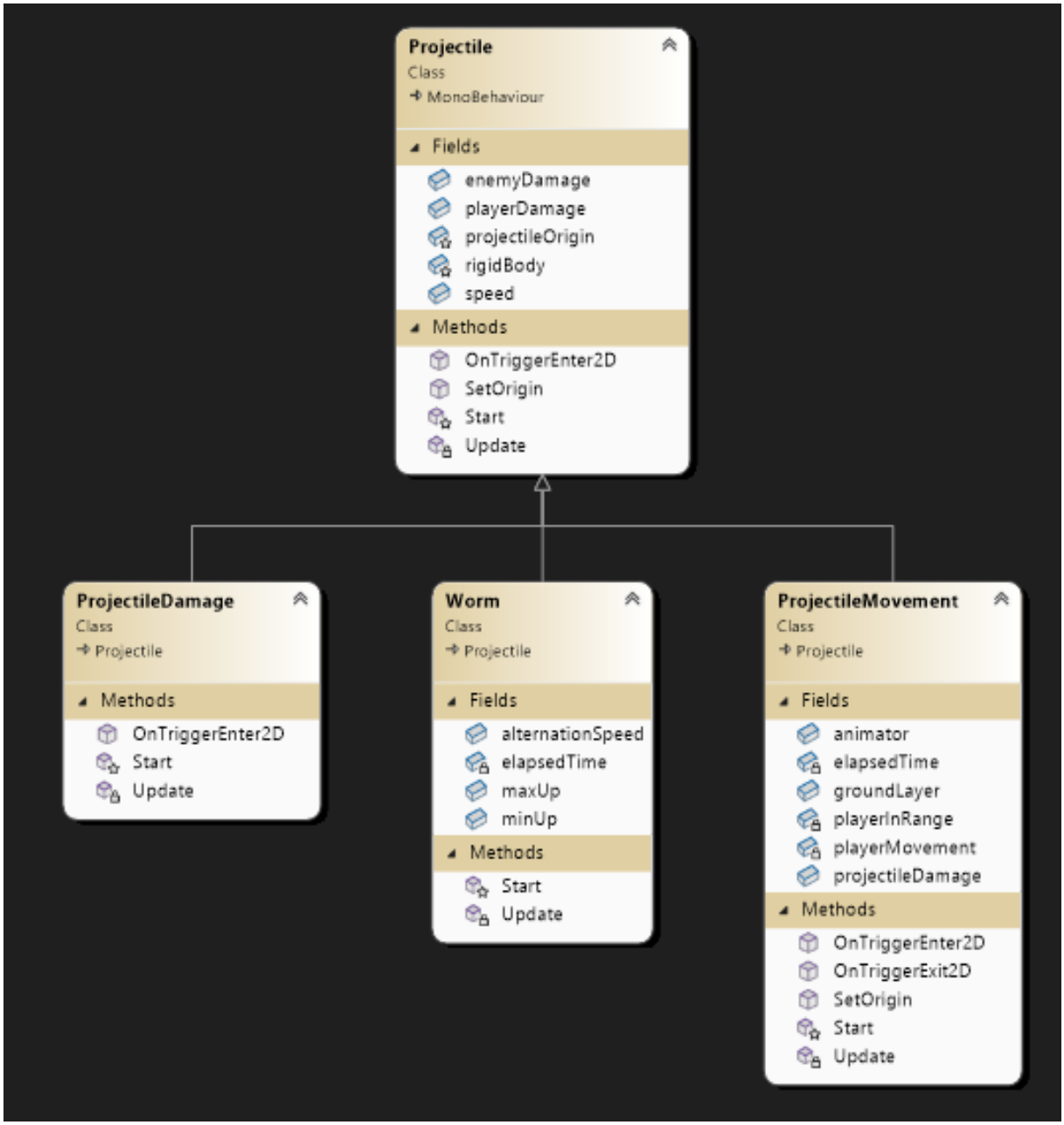
Appendix B: Trello board part 1

Appendix C: Trello board part 2

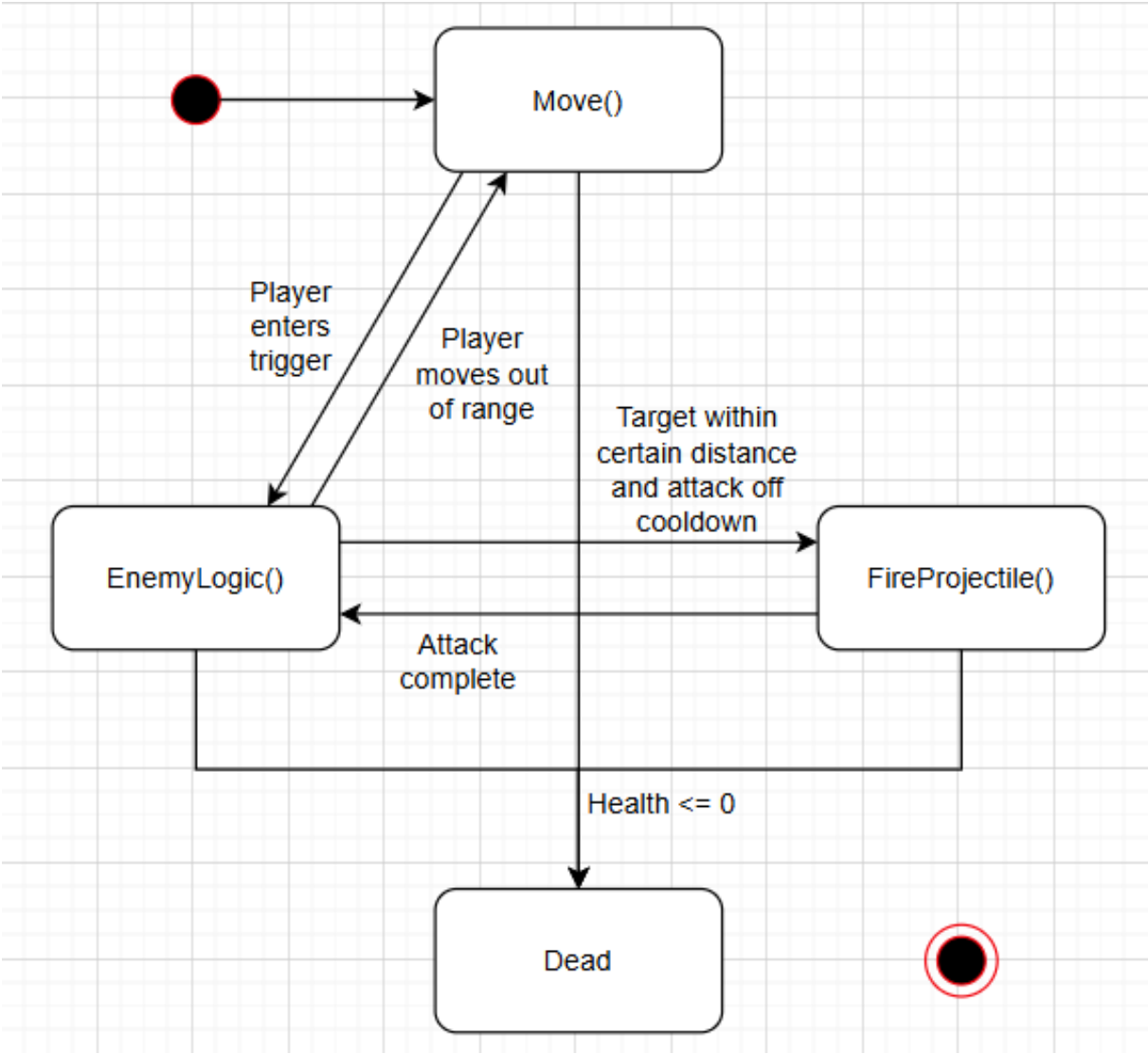


Appendix D: Trello board part 3

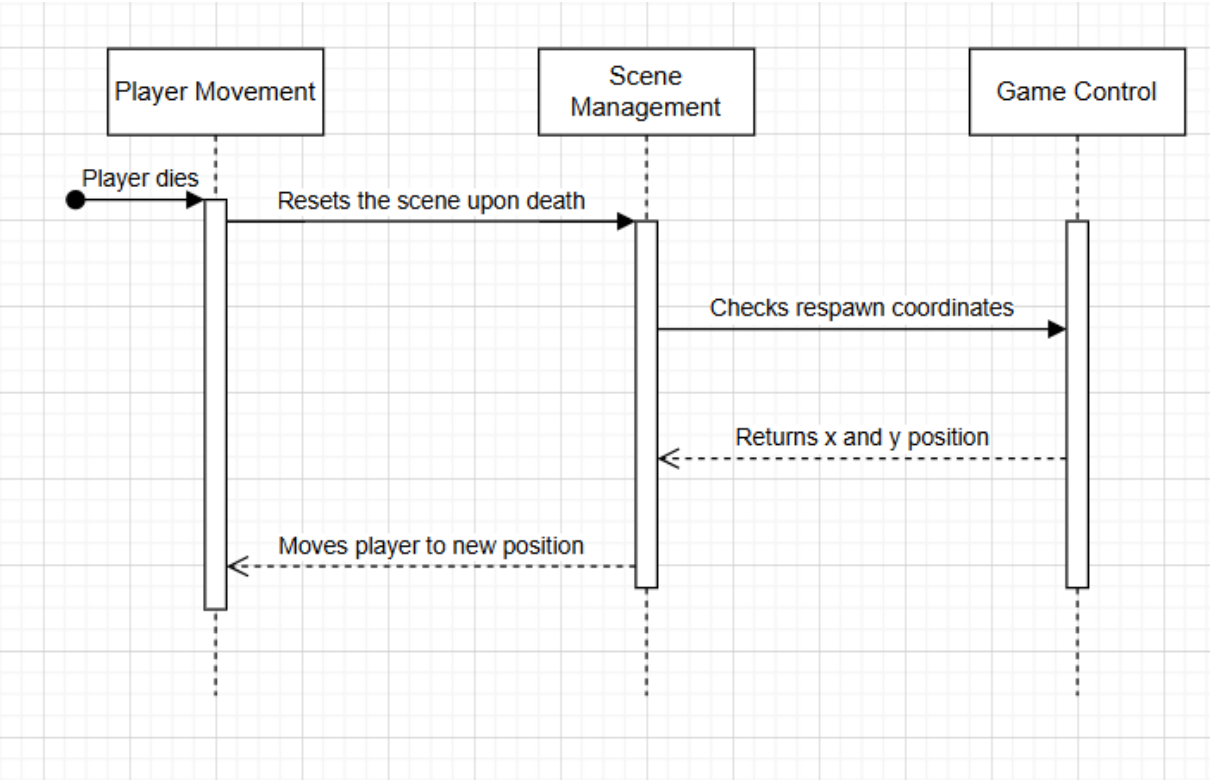
10.3 Technical Design & Architecture



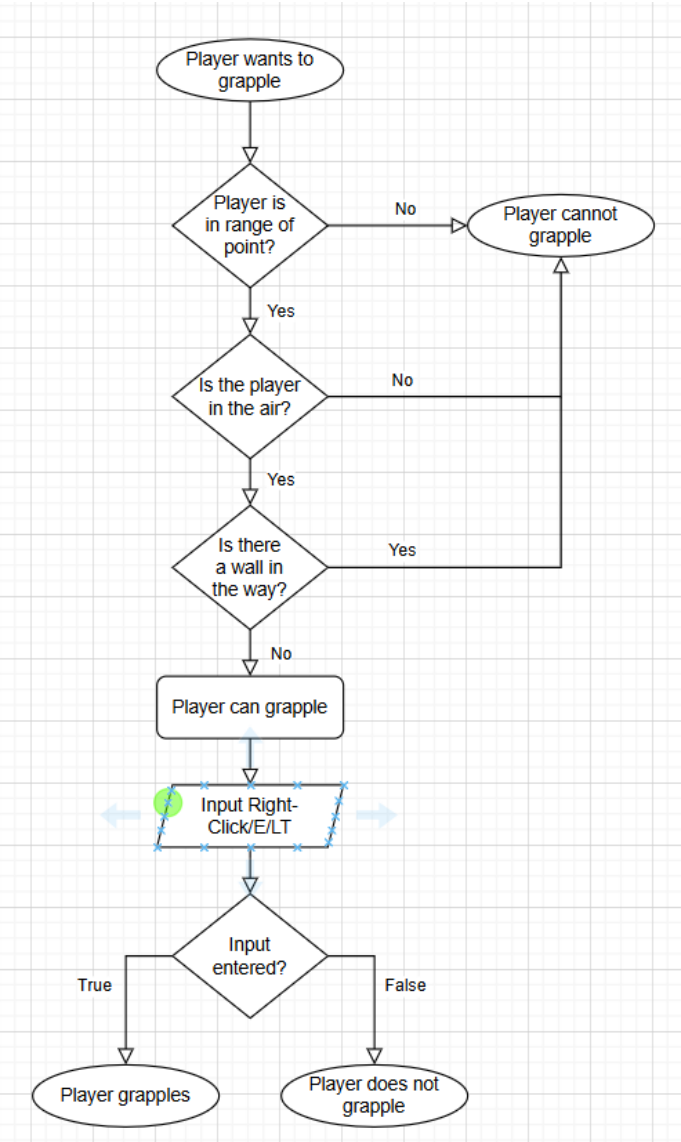
Appendix E: Class diagram of Projectile() and child classes



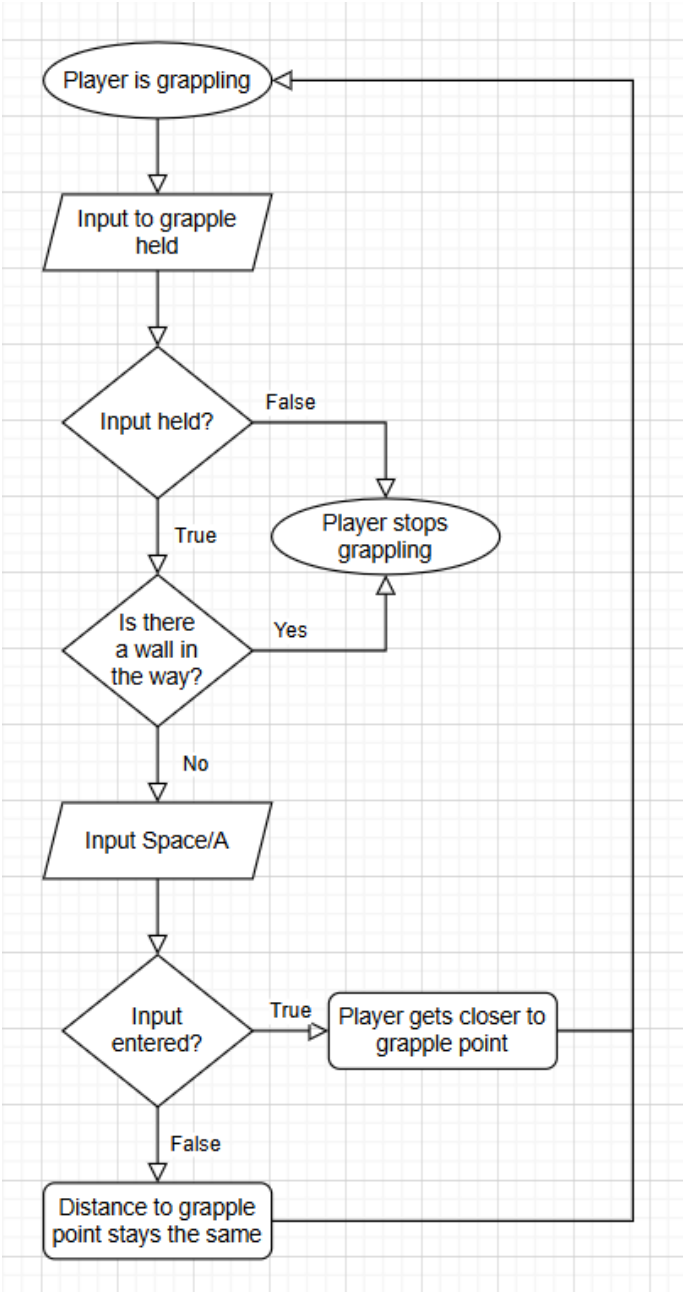
Appendix F: State machine of base enemy AI



Appendix G: Data flow diagram of load respawn coordinates upon death



Appendix H: Flow chart for checking if a player can grapple



Appendix I: Flow chart that checks if the player can continue grappling

10.4 Code Samples

10.4.1 Player Movement

```
if (isGrappling)
{
    // Rotates the player and changes the animation based on the player's position and direction relative to the grapple point.
    float angle = Mathf.Atan2((grapplePoint.position.y - transform.position.y), (grapplePoint.position.x - transform.position.x)) * Mathf.Rad2Deg;

    if ((rigidBody.linearVelocityX > 1 && transform.position.y <= grapplePoint.position.y) ||
        (rigidBody.linearVelocityX < 0 && transform.position.y > grapplePoint.position.y) ||
        (rigidBody.linearVelocityX == 0 && rigidBody.linearVelocityY < 0 && transform.position.x < grapplePoint.position.x))
    {
        if (angle < 22.5 && angle >= -22.5)
        {
            animator.SetFloat("swingDirection", -1f);
            transform.eulerAngles = new Vector3(0, 0, angle);
        }
        else if (angle <= 67.5 && angle >= 22.5)
        {
            animator.SetFloat("swingDirection", 0f);
            transform.eulerAngles = new Vector3(0, 0, angle - 45f);
        }
        else
        {
            animator.SetFloat("swingDirection", 1f);
            transform.eulerAngles = new Vector3(0, 0, angle - 90f);
        }
    }
    else if ((rigidBody.linearVelocityX < 1 && transform.position.y <= grapplePoint.position.y) ||
        (rigidBody.linearVelocityX > 0 && transform.position.y > grapplePoint.position.y) ||
        (rigidBody.linearVelocityX == 0 && rigidBody.linearVelocityY < 0 && transform.position.x > grapplePoint.position.x))
    {
        if (angle <= -157.5 || angle > 157.5)
        {
            animator.SetFloat("swingDirection", -1f);
            transform.eulerAngles = new Vector3(0, 180, -(angle + 180));
        }
        else if (angle >= 112.5 && angle <= 157.5)
        {
            animator.SetFloat("swingDirection", 0f);
            transform.eulerAngles = new Vector3(0, 180, -(angle - 135f));
        }
        else
        {
            animator.SetFloat("swingDirection", 1f);
            transform.eulerAngles = new Vector3(0, 180, -(angle - 90f));
        }
    }
}
```

Appendix J: Script to change the player's rotation based on the angle while grappling

```
void OnEnable()
{
    inputSystem.Player.Enable();
    inputSystem.Player.Move.performed += Movement;
    inputSystem.Player.Aim.performed += Aiming;
    inputSystem.Player.Jump.performed += Jumping;
    inputSystem.Player.Swing.performed += Grappling;
    inputSystem.Player.Attack.performed += Shooting;

    inputSystem.Player.Move.canceled += Movement;
    inputSystem.Player.Aim.canceled += Aiming;
    inputSystem.Player.Jump.canceled += Jumping;
    inputSystem.Player.Swing.canceled += Grappling;
    inputSystem.Player.Attack.canceled += Shooting;
}

void OnDisable()
{
    inputSystem.Player.Disable();
    inputSystem.Player.Move.performed -= Movement;
    inputSystem.Player.Aim.performed -= Aiming;
    inputSystem.Player.Jump.performed -= Jumping;
    inputSystem.Player.Swing.performed -= Grappling;
    inputSystem.Player.Attack.performed -= Shooting;

    inputSystem.Player.Move.canceled -= Movement;
    inputSystem.Player.Aim.canceled -= Aiming;
    inputSystem.Player.Jump.canceled -= Jumping;
    inputSystem.Player.Swing.canceled -= Grappling;
    inputSystem.Player.Attack.canceled -= Shooting;
}

void Movement(InputAction.CallbackContext context)
{
    // If movement has been inputted, this will determine the direction and speed of the player's movement.
    if (context.performed && Mathf.Abs(rigidBody.linearVelocityX) <= wallJumpSpeed.x)
    {
        isMoving = true;
        animator.SetBool("isMoving", true);
        movement = context.ReadValue<Vector2>().x;
    }
    // If movement is cancelled, player movement will stop and running animation will stop.
    else if (context.canceled && !isWallJumping)
    {
        movement = 0;
        isMoving = false;
        animator.SetBool("isMoving", false);
    }
    else if (context.canceled)
    {
        isMoving = false;
        animator.SetBool("isMoving", false);
    }
}
```

Appendix K: Initialisation of input system and base Movement function

10.4.2 Projectiles

```
public virtual void OnTriggerEnter2D(Collider2D collider)
{
    if (collider.gameObject == projectileOrigin)
    {
        return; // Ignore collision with the object that fired the projectile
    }
    // If the projectile hits the ground, destroy the projectile
    if (collider.gameObject.CompareTag("Ground"))
    {
        Destroy(gameObject);
    }
    else if (collider.gameObject.CompareTag("Enemy"))
    {
        // If the projectile hits an enemy, deal damage to the enemy and destroy the projectile
        FishMovement fishMovement = collider.GetComponent<FishMovement>();
        if (fishMovement != null)
        {
            fishMovement.TakeDamage(enemyDamage);
            // If the player damaged the enemy, the player becomes the enemy target.
            if (fishMovement.triggerArea.activeInHierarchy && projectileOrigin.CompareTag("Player"))
            {
                fishMovement.triggerArea.SetActive(false);
                fishMovement.target = projectileOrigin.transform;
                fishMovement.inRange = true;
                fishMovement.hotZone.SetActive(true);
                fishMovement.Flip();
            }
        }
```

Appendix L: Object detection when projectiles collide with different objects

```
public class Worm : Projectile
{
    public float minUp = -1f;
    public float maxUp = 1f;
    public float alternationSpeed = 2f;

    private float elapsedTime = 0f;

    // Start is called once before the first execution of Update after the MonoBehaviour is created
    protected override void Start()
    {
        base.Start();
    }

    // Update is called once per frame
    void Update()
    {
        // Moves the projectile up and down to simulate a worm-like movement.
        elapsedTime += Time.deltaTime;
        float offset = Mathf.Lerp(minUp, maxUp, (Mathf.Sin(elapsedTime * alternationSpeed) + 1) / 2);
        if (rigidBody.linearVelocityX < 0)
        {
            rigidBody.linearVelocity = (Vector2)(transform.right * speed) + (Vector2)(-transform.up * offset);
        }
        else
        {
            rigidBody.linearVelocity = (Vector2)(transform.right * speed) + (Vector2)(transform.up * offset);
        }
    }
}
```

Appendix M: Worm's modified Update() to oscillate in the air

10.4.3 Game Control

```
public void Save()
{
    // Saves the game data to a file.
    BinaryFormatter bf = new BinaryFormatter();
    using (FileStream file = File.Create(Application.persistentDataPath + "/gameInfo.dat"))
    {
        GameData data = new GameData();

        data.device = device;

        data.projectiles = projectiles;
        data.projectileIndex = projectileIndex;

        data.levelOneRespawnX = levelOneRespawnX;
        data.levelOneRespawnY = levelOneRespawnY;
        data.levelTwoRespawnX = levelTwoRespawnX;
        data.levelTwoRespawnY = levelTwoRespawnY;
        data.levelThreeRespawnX = levelThreeRespawnX;
        data.levelThreeRespawnY = levelThreeRespawnY;

        data.levelOneTimer = levelOneTimer;
        data.levelTwoTimer = levelTwoTimer;
        data.levelThreeTimer = levelThreeTimer;

        data.levelOneBest = levelOneBest;
        data.levelTwoBest = levelTwoBest;
        data.levelThreeBest = levelThreeBest;

        data.levelOneClear = levelOneClear;
        data.levelTwoClear = levelTwoClear;
        data.levelThreeClear = levelThreeClear;

        data.timePlayed = timePlayed;
        data.totalKills = totalKills;
        data.totalDeaths = totalDeaths;

        bf.Serialize(file, data);
        file.Close();
    }
}
```

Appendix N: The Save function of the game's saving system

10.4.4 Boss

```
private IEnumerator MineSpawner()
{
    // Teleports to the attack zone in the current phase and starts the mine spawning animation.
    // Animation called the SpawnMines function.
    invulnerable = true;
    if (phaseOne && Vector2.Distance(transform.position, phaseOneAttackZone.transform.position) > 2f)
    {
        animator.SetTrigger("disappear");
        yield return new WaitForSeconds(disappearDuration);

        transform.position = phaseOneAttackZone.transform.position;

        animator.SetTrigger("reappear");
        yield return new WaitForSeconds(reappearDuration);
    }
    else if (phaseTwo && Vector2.Distance(transform.position, phaseTwoAttackZone.transform.position) > 2f)
    {
        animator.SetTrigger("disappear");
        yield return new WaitForSeconds(disappearDuration);

        transform.position = phaseTwoAttackZone.transform.position;

        animator.SetTrigger("reappear");
        yield return new WaitForSeconds(reappearDuration);
    }
    else if (phaseThree && Vector2.Distance(transform.position, phaseThreeAttackZone.transform.position) > 2f)
    {
        animator.SetTrigger("disappear");
        yield return new WaitForSeconds(disappearDuration);

        transform.position = phaseThreeAttackZone.transform.position;

        animator.SetTrigger("reappear");
        yield return new WaitForSeconds(reappearDuration);
    }
    invulnerable = false;
    animator.SetTrigger("summonMines");
}
```

Appendix O: The boss's function that teleports them to a location, before spawning mines in a different function

10.5 User Testing / Feedback

10.5.1 Level Completion Rate

1. Were you able to successfully defeat the boss at the end and clear the level?

More details

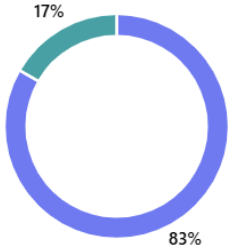
Yes	6
No	6

Appendix P: Question 1 of the usability test

10.5.2 Player Previous Experience

3. Do you have any experience playing 2d platforming games or any video games in general? More details

I have experience with both 2d platformers and other games.	10
I have experience with 2d platformers, but not with other games.	0
I have experience with video games, but not with any 2d platformers.	2
I do not have any experience with video games.	0
Other	0

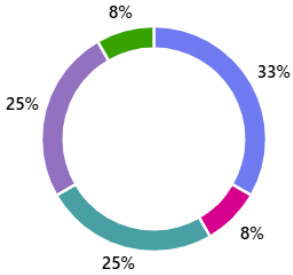


Appendix Q: Question 3 of the usability test

10.5.3 Alternate Bait Usage

4. Were you able to utilise any of the different kinds of bait, other than the default bait? More details

I used the other bait and I enjoyed using them	4
I used the other bait and I didn't enjoy using them	1
I didn't use any other bait because I didn't want to	3
I didn't use any other bait because I didn't know how	3
Other	1

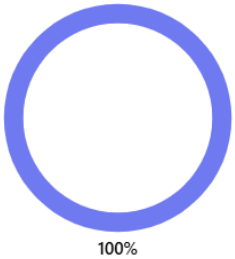


Appendix R: Question 4 of the usability test

10.5.4 Chance of Replayability

5. If given the option, would you be likely to replay level one to try and obtain a better time? More details

Yes	12
Maybe	0
No	0



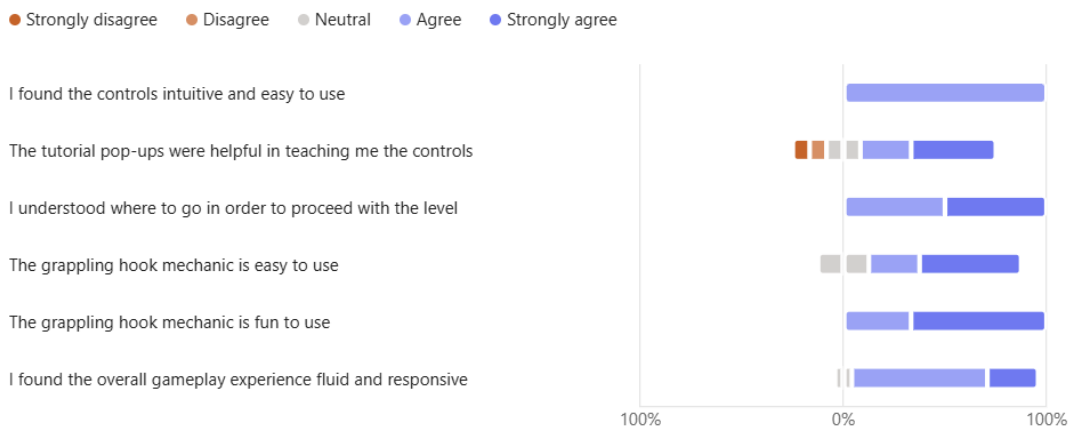
Appendix S: Question 5 of the usability test

4	anonymous	I want to try with different bait types
5	anonymous	Kinda messed around a bit and want try see how fast i can get
6	anonymous	It was fun
7	anonymous	I want to be the quickest
8	anonymous	Because it's a simple premise and feels very speedrunable
9	anonymous	Mechanics lend themselves well to going fast

Appendix T: Question 6 of the usability test

10.5.5 Likert Scale Questions

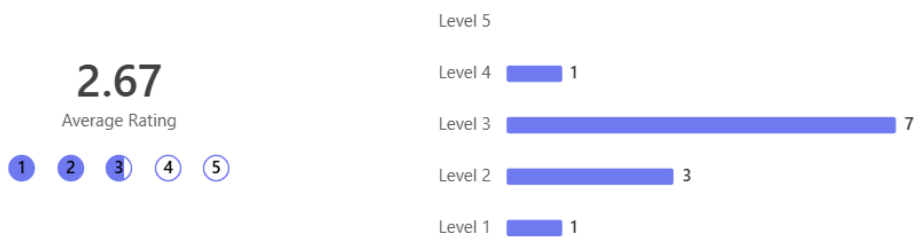
7. How much do you agree with the following statements about the gameplay and learning how to play? More details



Appendix U:Question 7 of the usability test

10.5.6 Difficulty Rating

8. On a scale of 1-5, 1 being too easy and 5 being too difficult, how would you rate the difficulty of the normal enemies? More details



Appendix V: Question 8 of the usability test

9. On a scale of 1-5, 1 being too easy and 5 being too difficult, how would you rate the difficulty of the boss, Minehead? More details

Appendix W: Question 9 of the usability test

10.5.7 Written Feedback

11. What are some features you liked about the game?

10 Responses

ID ↑	Name	Responses
1	anonymous	The grappling and speed mechanics
2	anonymous	the fishing hook, the game of it being underwater
3	anonymous	I like the way you can feel yourself up with the fishing rod
4	anonymous	Grappling hook and movement in general
5	anonymous	It was fun to swing around and try different bait types
6	anonymous	Grappling is very fun
7	anonymous	Hook mechanic was really good
8	anonymous	The grappling mechanic and secret rooms were fun
9	anonymous	I like the platforming and scene changes during the boss fight. the pixel art is well done, the movement with the hook feels smooth and does what you expect it to. controls are well explained and follow gaming standards. I appreciated the secrets sections like finding the bait. the name is creative and hooks you in 😉
10	anonymous	Wall jumping and grapple

Appendix X: Question 11 of the usability test

12. Are there any changes you would make to the game?

11 Responses

ID ↑	Name	Responses
1	anonymous	Make walljumps a little more generous, mistining can kill you
2	anonymous	maybe alter some of the pixel arts for the walls
3	anonymous	Add a health bar for the small enemies
4	anonymous	Grappling allow me to build momentum
5	anonymous	Movement not stopping your momentum
6	anonymous	Improve the feel of controller gameplay
7	anonymous	Boss felt a bit easy
8	anonymous	Add music
9	anonymous	Make it longer

Appendix Y:Question 12 of the usability test part 1

10	anonymous	healing at checkpoint or via some other method would be nice. A quick dying animation/game over screen might be nice. music +sound effects would be nice but not necessary, I feel like wall jumping on a single wall is too powerful. Make damage feel more weighty with feedback for both taking damage by a enemy and when hitting an enemy. I feel like increasing bait asset size would be useful, as they feel a bit small if possible. When changing baits, it's not communicated when you've selected another bait and not shown/explained how to exit. Fix the mines not doing damage during the boss. When doing the platforming part on the boss, the transition to platform to next stage feels a bit jarring going into, I think, the second stage. Some variety in the boss fight in terms of abilities/attack patterns like an enraged mode when low health or variety in normal enemies would be nice if have time, E.g some vertically for normal enemies.The boss feels like it moves too often and too randomly to set sections, this might feel better if the mines are fixed.
11	anonymous	Menus could have a clear way to show when hovering over the buttons. Some controls could be improved like the bait screen could have the exit button be "B" which is the standard.

Appendix Z: Question 12 of the usability test part 2

13. Did you experience any bugs or technical issues? If yes, what were they?

10 Responses

ID ↑	Name	Responses
1	anonymous	Nope!
2	anonymous	sometimes the ball clips through the ground
3	anonymous	nope
4	anonymous	First set of spikes, ground detection bug
5	anonymous	No bugs
6	anonymous	One of the breakable walls did not reveal the path
7	anonymous	Yes wall not being removed and glitch stuck on spikes
8	anonymous	The mines did not damage me, the tutorial tips were for m+k only but I played on controller, and I could not change bait on controller
9	anonymous	mines in boss right not working
10	anonymous	Can jump up to the higher platform in the boss before getting to the second phase.

Appendix AA: Question 13 of the usability test

10.6 Asset & Resource List

Background of level one and two, UnderWater Diving made by Ansimuz found on the Unity Asset Store: https://assetstore.unity.com/packages/2d/environments/underwater-diving-94029

Tileset used in level one, SpearFishing asset pack made by Szym found on itch.io: https://nszym.itch.io/spearfishing-assets-pack

Background of main menu, Parallax Backgrounds: Dock made by Admurin found on itch.io: https://admurin.itch.io/parallax-backgrounds-dock

Font for title on main menu, Zain Fishing Hooks Font uploaded by audreanne87 on font.download: https://font.download/font/zain-fishing-hooks

10.7 Risk Assessment

Risk	Who/what it will affect	Likelihood score	Impact score	Total risk score	Mitigation	New likelihood score	New impact score	New total score
Loss of device or report.	No report will be available to present.	1/5	5/5	5/25	Game saved and uploaded	1/5	2/5	2/25

					to GitHub repository.			
Game not finished in time.	No report will be available to present.	2/5	5/5	10/25	Sprints will be planned before to find the MVP.	1/5	5/5	5/25
Game breaking bug found on release.	Game might not be playable due to bug not found before.	3/5	5/5	15/25	Usability testing will be held in stages before release.	1/5	5/5	5/25
Copyright issues.	Loss of marks and removal of game from stores if the correct licensing has not been acquired.	2/5	4/5	8/25	Licensing checked before.	1/5	4/5	4/25
Difficulty spike and steep learning curve.	Players quit due to difficulty of the game.	3/5	4/5	12/25	Usability testing held before release and enemy damage and platforming will be tweaked as needed.	1/5	4/5	4/25
Game priced too high.	Players that want to play the game cannot because the game is too expensive.	2/5	4/5	8/25	Game will be priced minimally to garner a fanbase and there will be no in-game purchases.	1/5	4/5	4/25

No marketing	Lack of players as few people will know about the game.	4/5	3/5	12/25	Game will come with a poster and product pitch, and a trailer might be made for a full release.	2/5	3/5	6/25
Managing bugs poorly.	Little time left for fixing bugs before release and so some bugs appear in the final product.	2/5	5/5	10/25	Testing performed throughout the game's creation and fixes can be made after release.	1/5	3/5	3/25
Game runs poorly.	People cannot play the game at a stable FPS, causing most people to quit.	3/5	4/5	12/25	Game will be a low poly pixel art game, so the game should run smoothly on low-end PC's.	1/5	4/5	4/25

Appendix BB: Risk assessment table

10.8 AI Declaration

Solo Work	**S1 - Generative AI tools have not been used for this assessment.**	☐
Assisted Work	**A1 – Idea Generation and Problem Exploration** Used to generate project ideas, explore different approaches to solving a problem, or suggest features for software or systems. Students must critically assess AI-generated suggestions and ensure their own intellectual contributions are central.	☐
	A2 - Planning & Structuring Projects AI may help outline the structure of reports, documentation and projects. The final structure and implementation must be the student's own work.	☒
	A3 – Code Architecture AI tools maybe used to help outline code architecture (e.g. suggesting class hierarchies or module breakdowns). The final code structure must be the student's own work.	☐
	A4 – Research Assistance Used to locate and summarise relevant articles, academic papers, technical documentation, or online resources (e.g. Stack Overflow, GitHub discussions. The interpretation and integration of research into the assignment remain the student's responsibility.	☐
	A5 - Language Refinement Used to check grammar, refine language, improve sentence structure in documentation not code. AI should be used only to provide suggestions for improvement. Students must ensure that the documentation accurately reflects the code and is technically correct.	☒
	A6 – Code Review AI tools can be used to check comments within the code and to suggest improvements to code readability, structure or syntax. AI should be used only to provide suggestions for improvement. Students must ensure that the code accurately reflects their knowledge and is technically correct.	☒
	A7 - Code Generation for Learning Purposes Used to generate example code snippets to understand syntax, explore alternative implementations, or learn new programming paradigms. Students must not submit AI-generated code as their own and must be able to explain how it works.	☒
	A8 - Technical Guidance & Debugging Support AI tools can be used to explain algorithms, programming concepts, or debugging strategies. Students may also help interpret error messages or suggest possible fixes. However, students must write, test, and debug their own code independently and understand all solutions submitted.	☐
	A9 - Testing and Validation Support AI may assist in generating test cases, validating outputs, or suggesting edge cases for software testing. Students are responsible for designing comprehensive test plans and interpreting test results.	☐
	A10 - Data Analysis and Visualization Guidance AI tools can help suggest ways to analyse datasets or visualize results (e.g.	☐

	recommending chart types or statistical methods). Students must perform the analysis themselves and understand the implications of the results.	
	A11 - Other uses not listed above Please specify:	☐
Partnered Work	**P1 - Generative AI tool usage has been used integrally for this assessment** Students can adopt approaches that are compliant with instructions in the assessment brief. Please Specify:	☐

Please provide details of AI usage and which elements of the coursework this relates to: In the documentation, AI was used as a grammar checker, a language refinement tool, and gave help with the initial structuring of the report, alongside the recommended sections in the guidance document. In the code of my project, AI was used to help with the issues I presented to it and it would tell me what was wrong based on the code and error messages I gave it and how to fix it with pseudocode.

I understand that the ownership and responsibility for the academic integrity of this submitted assessment falls with me, the student.	☒
I confirm that all details provide above are an accurate description of how AI was used for this assessment.	☒